

# Offline Shop Management App — Full Updated Specification

## 1. Purpose

Build an offline shop management app that works without internet.

The system uses:

- 1 laptop
- 1 mobile phone
- 1 Bluetooth module, such as HC-05
- PostgreSQL database on or accessible by the laptop
- Qt desktop application on the laptop
- Mobile app on the phone

There is **no backend server**.

The Qt app reads and writes directly to PostgreSQL.

The mobile app does **not** connect directly to PostgreSQL. It works offline using its own local database and syncs with the Qt app through Bluetooth.

---

## 2. System Overview

### 2.1 Main Architecture

PostgreSQL

↑↓

Qt Laptop App

↑↓ Bluetooth / HC-05

Mobile App

### 2.2 Main Responsibilities

**Qt Laptop App**

The Qt app is the main system owner.

It manages:

- user login
- admin user management
- master product information
- official stock quantity
- stock transaction history
- unknown product review
- stock addition confirmation
- stock removal confirmation
- physical inventory count review
- stock adjustment
- Bluetooth sync with mobile
- PostgreSQL read/write

## **Mobile App**

The mobile app is mainly used for fast shop operation.

It supports:

- offline login
- QR login
- barcode scanning
- product picture capture for local unknown product reference only
- local product lookup
- temporary stock counting
- stock addition
- stock removal
- physical inventory counting
- unknown product list
- pending sync
- Bluetooth sync with Qt

## **PostgreSQL Database**

PostgreSQL stores the official data:

- users
- products
- stock quantities
- stock transactions
- unknown product records

physical count sessions  
sync history  
login token records

---

## 3. Source of Truth Rules

The Qt app and PostgreSQL are the source of truth.

The mobile app stores a local copy for offline work.

### 3.1 Qt Owns

official product information  
official user accounts  
official stock quantity  
official stock transaction history  
official physical count review result  
stock adjustments

### 3.2 Mobile Temporarily Stores

local product copy  
local user login copy  
unknown products  
unknown product pictures stored locally only  
pending stock additions  
pending stock removals  
pending physical count sessions  
pending product info suggestions  
pending sync batches

### 3.3 Important Rule

The mobile app should never directly change official stock quantity.

The mobile app only sends scanned/count data to Qt.

Qt confirms and saves the final official result.

---

## 4. No Backend Rule

The system will not have a backend server.

Therefore:

Qt app connects directly to PostgreSQL.

Mobile app does not connect directly to PostgreSQL.

Mobile app communicates with Qt only through Bluetooth.

The mobile app must not store PostgreSQL credentials.

The mobile app must not attempt to connect to PostgreSQL through Bluetooth.

---

## 5. User Login and User Management

### 5.1 Purpose

Login is required so every stock operation can record who performed it.

Every important action must record:

user\_id  
user\_name\_snapshot  
timestamp  
device\_id

This applies to:

stock addition  
stock removal  
stock adjustment  
physical inventory count  
product information update  
unknown product resolution  
user management action  
sync action

---

### 5.2 User Management Rule

Users can be added, updated, deactivated, or deleted only in the Qt app Admin Mode.

The mobile app cannot create users.

The mobile app cannot delete users.

The Qt app is the source of truth for users.

---

## 5.3 User Roles

The system should support at least:

admin  
staff

### Admin Can

login to Qt  
access Admin Mode  
create users  
update users  
deactivate users  
delete users if allowed  
reset passwords  
sync users to mobile  
manage product information  
review unknown products  
add stock  
remove stock  
review physical inventory count  
create stock adjustments  
view transaction history

### Staff Can

login to Qt or mobile  
scan products  
add products to stock  
remove products from stock  
perform physical inventory count  
send pending sync data

### Staff Cannot

- access Admin Mode
- create users
- delete users
- reset other users' passwords
- delete transaction history
- directly change product master data unless allowed

---

## 6. User Registration

When an admin creates a user in Qt Admin Mode, the admin should enter:

- user\_id
- name
- password
- birthday
- mobile\_number
- role
- status

Recommended user fields:

- user\_id
- name
- password\_hash
- password\_salt
- birthday
- mobile\_number
- role
- status
- created\_at
- updated\_at
- created\_by
- updated\_by
- last\_login\_at

### 6.1 Password Rule

Plain passwords must never be stored.

Plain passwords must never be sent through Bluetooth.

Passwords should be stored as a secure password hash.

Recommended:

Argon2id

Acceptable for MVP:

bcrypt

Do not use plain SHA256 without salt.

---

## 7. User Delete and Deactivate Rule

Users should not always be physically deleted.

If a user has already created stock transactions, deleting that user would damage transaction history.

Example problem:

Transaction:

user\_id = staff001

But user staff001 was deleted.

Now the history does not clearly show who performed the action.

Recommended rule:

If a user has no transaction history, admin may delete the user.

If a user has transaction history, admin should deactivate the user instead of deleting.

A deactivated user:

cannot login

remains visible in old transaction history

can be reactivated by admin if needed

---

## 8. Qt Login

The Qt app should support login using:

user\_id  
password

Qt login flow:

1. User opens Qt app.
2. User enters user\_id and password.
3. Qt reads the user record from PostgreSQL.
4. Qt verifies the password hash.
5. Qt checks that user status is active.
6. If valid, login succeeds.
7. Qt stores the current logged-in user in the app session.
8. Qt displays pages based on the user's role.

Admin users can access Admin Mode.

Staff users cannot access Admin Mode.

---

## 9. Mobile Login

The mobile app should support:

user\_id + password login  
QR code login

Because the mobile app works offline, it must use synced local user data.

The mobile app should not connect to PostgreSQL.

---

### 9.1 Mobile Login by User ID and Password

Mobile login flow:

1. User enters user\_id and password.
2. Mobile checks its local synced user database.
3. Mobile verifies password using local login verifier or password hash.
4. Mobile checks that user status is active.
5. If valid, login succeeds.
6. Mobile stores the current logged-in user in the app session.



The mobile app must not store plain passwords.

---

## 9.2 Mobile Login by QR Code

After a user logs in, the system can generate a QR code for faster login.

The QR code must not contain sensitive full user information.

The QR code must not contain:

- plain password
- password hash
- birthday
- mobile number

Bad QR content:

- user\_id
- name
- password
- birthday
- mobile\_number

Recommended QR content:

- user\_id
- login\_token
- expires\_at
- signature

For MVP, QR code can contain:

- user\_id
- short-lived login token
- expires\_at

The QR login token should expire.

Recommended MVP rule:

QR login token expires after 24 hours.

---

## 10. User Sync Between Qt and Mobile

When admin adds, updates, deactivates, deletes, or resets password for a user, the Qt app should activate:

Send User Data to Mobile

The Qt app should show:

User information was updated. Please sync users with the mobile app.

Synced user data should include only what mobile needs:

user\_id  
name  
role  
status  
login verifier or password hash  
updated\_at  
sync\_version

Recommended:

Qt stores birthday and mobile number.  
Mobile does not need birthday and mobile number.

---

## 11. Product Information

Product information belongs to one barcode.

Product information should be separate from stock transaction price.

### 11.1 Product Fields

Recommended fields:

barcode  
product\_name  
country\_of\_manufacture  
product\_type  
default\_price  
details

created\_at  
updated\_at  
created\_by  
updated\_by  
sync\_version

## 11.2 Required Product Fields

Before a product can be finalized into official stock, required fields are:

barcode  
product\_name  
product\_type

Stock-entry price is required for stock value calculation.

The default product price may be optional if stock-entry price is provided during stock addition.

## 11.3 Optional Product Fields

country\_of\_manufacture  
default\_price  
description  
supplier note  
local mobile picture for unresolved unknown products only  
other details

---

# 12. Product Default Price vs Stock-Entry Price

The system must separate:

product default price

from:

stock-entry price

## 12.1 Product Default Price

The default price is stored in product information.

It is the normal product price.

## 12.2 Stock-Entry Price

The stock-entry price is entered when products are added to stock.

It is the price for that specific stock addition batch.

Example:

May 1:

barcode 123456

quantity 10

stock-entry price 5.00

May 3:

barcode 123456

quantity 20

stock-entry price 6.00

Same product, different stock-entry price.

The stock-entry price should be used for stock value calculation for that batch.

Changing the stock-entry price should not automatically overwrite the product default price.

---

# 13. Product Scanning Workflow

## 13.1 Known Product

When the user scans a barcode on mobile:

1. Mobile searches local product database.
2. If product exists, mobile displays product information.
3. User can continue stock operation.

Displayed information may include:

product name

country of manufacture  
product type  
default price  
details

Known products do not require product pictures.

Known product pictures should not be sent to Qt.

---

## 13.2 Unknown Product

If mobile cannot find product information for a barcode:

1. Mobile adds the barcode to Unknown List.
2. Mobile allows user to take product picture for local reference only.
3. Mobile does not send the product picture to Qt.
4. Mobile allows user to enter temporary product info if desired.
5. Mobile still allows quantity counting.

Unknown List should contain only one item per barcode.

If the same unknown barcode is scanned again, update the existing item instead of creating duplicates.

---

# 14. Unknown List

## 14.1 Unknown List Fields

Each Unknown List item should store:

barcode  
local\_product\_picture  
first\_seen\_at  
last\_seen\_at  
temporary\_product\_name  
temporary\_price  
pending\_quantity  
sync\_status

The product picture stays only on the mobile app.

The picture is not synced to Qt.

## 14.2 Product Picture Rule

Mobile should store product pictures only for unresolved unknown products.

For each unknown barcode, mobile should keep only one representative picture.

If the user takes a new picture for the same unknown barcode, replace the old picture.

After product information is completed in Qt and synced from Qt to mobile:

mobile removes barcode from Unknown List

mobile deletes local picture for that barcode

mobile keeps only product information

## 14.3 Barcode Matching Rule for Unknown Products

Qt should display the barcode clearly as the main header of each unknown item.

The user should compare:

Qt Unknown List barcode

with:

Mobile Unknown List barcode and local picture

This allows the user to enter product information in Qt by looking at the correct product picture on mobile without transferring images through Bluetooth.

---

# 15. Sending Unknown List to Qt

When the user clicks:

Send Unknown List

the mobile app should send unresolved unknown product data to Qt through Bluetooth.

The mobile app should send only barcode and related text/quantity metadata.

The mobile app should not send product pictures to Qt.

Data should include:

sync\_batch\_id  
barcode  
temporary\_product\_name  
temporary\_price  
first\_seen\_at  
last\_seen\_at  
pending\_quantity  
created\_by\_user\_id  
device\_id

The Qt app should display the Unknown Product Review page.

Each item should show:

barcode as the main header  
temporary information if available  
pending counted quantity if available  
first seen time  
last seen time  
sync status

The Qt user can then compare the barcode with the barcode and local picture shown on the mobile app.

The Qt user can then enter official product information.

---

## 16. Completing Product Information in Qt

In Qt, the user can enter:

product name  
country of manufacture  
product type  
default price  
details

After saving:

Qt marks the unknown barcode as resolved.

Qt creates or updates official product information.  
Qt links pending stock counts to the product.  
Qt activates Send Product Info to Mobile.

The Qt app should show:

You updated some product information. Please sync it with the mobile app.

---

## 17. Sending Product Info to Mobile

When the user clicks:

Send Product Info to Mobile

Qt sends updated product information to mobile through Bluetooth.

After mobile receives and saves it:

mobile updates local product database  
mobile removes resolved barcodes from Unknown List  
mobile deletes local pictures for resolved barcodes  
mobile links pending stock counts to the updated product info

---

## 18. Mobile Product Info Entry

The mobile app should provide a UI for entering or updating product information without Qt.

This is useful when the laptop is not available.

However, mobile-entered or mobile-updated product information is temporary.

Mobile may allow:

product name  
country of manufacture  
product type  
default price  
details

When a user adds or updates product information on mobile:



1. Mobile saves the change locally as a temporary product info suggestion.
2. Mobile marks the suggestion as Pending Sync.
3. When Bluetooth becomes available, mobile syncs the suggestion to Qt.
4. Qt shows the suggestion for review.
5. Qt user can accept, edit, or reject the suggestion.
6. If accepted, Qt saves it as official product information in PostgreSQL.
7. Qt sends the confirmed product information back to mobile.
8. Mobile replaces the temporary suggestion with the official product information.

For MVP:

Qt is the final authority for product information.

Mobile product info is only a suggestion until accepted by Qt.

---

## 19. Stock Quantity and Stock Transactions

The system should not only update current stock quantity.

Every stock movement must create a transaction record.

Transaction types:

stock\_addition  
stock\_removal  
stock\_adjustment

Adjustment reasons may include:

physical\_inventory\_count  
manual\_correction  
damage\_correction  
other

Every transaction should include:

transaction\_id  
transaction\_type  
batch\_id  
barcode  
quantity  
unit\_price if applicable  
source\_location if applicable  
supplier\_name if applicable

carried\_by if applicable  
reason if applicable  
destination\_type if applicable  
destination\_name if applicable  
destination\_note if applicable  
user\_id  
user\_name\_snapshot  
transaction\_time  
device\_id  
sync\_status  
created\_at

---

## 20. Adding Products to Stock

### 20.1 Purpose

Adding products to stock means new products came into the shop.

It increases official stock quantity after confirmation.

The app should support:

barcode scanning  
repeated quantity entry  
stock-entry price input  
source information  
carried-by information  
offline counting  
Bluetooth sync  
Qt confirmation

---

### 20.2 Adding Stock When Bluetooth Is Connected

When Bluetooth is connected:

1. User logs in.
2. User scans barcode using mobile.
3. Mobile sends scan event to Qt in real time.
4. Qt displays temporary counted quantity for the barcode.
5. Each additional scan increases temporary quantity.

6. User can enter repeated quantity instead of scanning one by one.
7. User enters stock-entry price for that batch.
8. User may enter source information.
9. User confirms in Qt by clicking “Add Products into Stock”.
10. Qt validates the product and quantity.
11. Qt saves stock addition transaction.
12. Qt increases official stock quantity.
13. Qt records user\_id and transaction time.
14. Qt sends ACK.
15. Temporary count is reset.

In this workflow:

Mobile = scanner

Qt = final confirmation and official stock owner

---

## 20.3 Adding Stock When Bluetooth Is Not Connected

When Bluetooth is not connected:

1. User logs in on mobile.
  2. User scans barcode.
  3. Mobile searches local product database.
  4. If product exists, mobile displays product info.
  5. If product does not exist, mobile adds it to Unknown List.
  6. User enters quantity or repeated quantity.
  7. User enters stock-entry price.
  8. User may enter source information.
  9. Mobile saves stock addition as Pending Sync.
  10. Later, when Bluetooth is available, mobile sends pending stock addition to Qt.
  11. Qt validates and saves it.
  12. Qt sends ACK.
  13. Mobile marks batch as synced.
  14. Mobile resets pending quantity only after ACK.
- 

## 20.4 Unknown Product Stock Addition

Unknown products can be counted.

Unknown products can be stored as pending stock additions.

But unknown products cannot be finalized into official stock inventory until required product information is completed.

Rule:

Unknown barcode can be counted.

Unknown barcode can have temporary stock-entry price.

Unknown barcode can be synced to Qt.

Unknown product picture stays only on mobile and is not sent to Qt.

Qt must complete product information before official stock addition is finalized.

---

## 20.5 Repeated Quantity Entry

The user can enter repeated quantity instead of scanning one by one.

Example:

barcode: 123456

quantity: 50

Quantity must be greater than zero.

---

## 20.6 Optional Stock Source Fields

When adding products to stock, user may enter:

source\_location

supplier\_name

carried\_by

purchase\_note

delivery\_note

These fields should be stored in stock transaction history.

---

# 21. Removing Products from Stock

## 21.1 Purpose

Removing products from stock means products leave the shop or should be deducted.

Examples:

sold  
damaged  
lost  
returned to supplier  
manual correction  
other

When removing products from stock, the system should also allow the user to specify the outgoing destination.

This is important because the reason and destination are different.

Example:

reason = returned to supplier  
destination = ABC Supplier Warehouse

reason = sold  
destination = Customer / Shop Sale

reason = transferred  
destination = Branch Store 2

reason = damaged  
destination = Damaged Goods Area

Recommended outgoing destination fields:

destination\_type  
destination\_name  
destination\_contact  
destination\_note

Recommended `destination_type` values:

customer  
supplier  
other\_shop  
warehouse  
damaged\_goods\_area  
lost\_unknown  
internal\_use

other

For MVP, the app can keep this simple:

removal\_reason  
destination\_name  
note

---

## 21.2 Removing Stock When Bluetooth Is Connected

1. User logs in.
  2. User scans barcode using mobile.
  3. Mobile sends scan event to Qt.
  4. Qt displays temporary removal quantity.
  5. User enters repeated quantity if needed.
  6. User selects removal reason.
  7. User enters outgoing destination information if applicable.
  8. User confirms removal in Qt.
  9. Qt checks current stock quantity.
  10. Qt prevents negative stock for MVP.
  11. Qt saves stock removal transaction.
  12. Qt decreases official stock quantity.
  13. Qt records user\_id and transaction time.
  14. Qt sends ACK.
  15. Temporary removal count is reset.
- 

## 21.3 Removing Stock When Bluetooth Is Not Connected

1. User logs in on mobile.
  2. User scans barcode.
  3. User enters quantity.
  4. User selects removal reason.
  5. User enters outgoing destination information if applicable.
  6. Mobile saves removal as Pending Sync.
  7. Later, mobile sends removal to Qt.
  8. Qt validates stock quantity.
  9. Qt saves removal transaction.
  10. Qt decreases official stock quantity.
  11. Qt sends ACK.
  12. Mobile marks removal as synced.
-

## 21.4 Outgoing Destination Rule

For every stock removal, the app should support optional outgoing destination information.

The destination explains where the product went after leaving stock.

Destination examples:

customer  
supplier  
another shop  
warehouse  
damaged goods area  
internal use location  
unknown / lost  
other

The destination should be stored in the stock removal transaction.

Recommended fields:

destination\_type  
destination\_name  
destination\_contact  
destination\_note

For MVP, only this may be required:

destination\_name

The **destination\_name** may be optional for reasons such as lost or damaged, but it should be available when useful.

---

## 21.5 Negative Stock Rule

For MVP:

Official stock quantity must not go below zero.

If user tries to remove more than available, Qt should show:

Not enough stock available.

---

## 22. Physical Inventory Count

### 22.1 Purpose

Physical Inventory Count is used to count the products that currently exist in the shop.

This is different from adding or removing stock.

Example:

System quantity: 100

Real counted quantity: 97

Difference: -3

Physical Inventory Count itself does not change stock.

Only a confirmed stock adjustment changes stock.

---

### 22.2 Difference Between Stock Operations

| Feature                    | Meaning                           | Changes Stock Immediately? |
|----------------------------|-----------------------------------|----------------------------|
| Add Products to Stock      | New products came into shop       | Yes, after confirmation    |
| Remove Products from Stock | Products left shop                | Yes, after confirmation    |
| Physical Inventory Count   | Count what currently exists       | No                         |
| Stock Adjustment           | Fix official stock based on count | Yes, after confirmation    |

Important rule:

Physical Inventory Count creates a count result.

Stock Adjustment changes official stock quantity.

---

### 22.3 Physical Count When Bluetooth Is Connected

1. User logs in.



2. User opens Physical Inventory Count page.
  3. User scans product barcode using mobile.
  4. Mobile sends scan event to Qt in real time.
  5. Qt increases temporary physical count quantity.
  6. User can enter repeated quantity.
  7. User completes counting.
  8. Qt compares counted quantity with official stock quantity.
  9. Qt shows matched, shortage, or overage result.
  10. User reviews the difference.
  11. User confirms whether to create stock adjustment.
  12. If confirmed, Qt creates stock adjustment transaction.
  13. Qt updates official stock quantity.
  14. Qt records user\_id and transaction time.
- 

## 22.4 Physical Count When Bluetooth Is Not Connected

1. User logs in on mobile.
2. User opens Physical Inventory Count page.
3. User scans products.
4. Mobile stores counted quantities locally.
5. Count session is saved as Pending Sync.
6. Later, when Bluetooth is available, mobile sends count session to Qt.
7. Qt compares counted quantities with official stock quantities.
8. Qt shows differences.
9. User confirms whether to create adjustments.
10. Qt creates stock adjustment transactions only after confirmation.
11. Qt sends ACK to mobile.
12. Mobile marks count session as synced.

Mobile must not directly change official stock quantity.

---

## 22.5 Physical Count Session

A physical inventory count should be stored as a session.

Session fields:

count\_session\_id  
started\_by\_user\_id  
started\_by\_user\_name\_snapshot  
started\_at

ended\_at  
status  
device\_id  
sync\_batch\_id  
note

Status values:

draft  
pending\_sync  
synced  
reviewed  
applied  
cancelled

Each counted item should store:

count\_session\_item\_id  
count\_session\_id  
barcode  
counted\_quantity  
system\_quantity\_at\_review  
difference\_quantity  
review\_status

Review status values:

matched  
shortage  
overage  
unknown\_product

---

## 22.6 Physical Count and Unknown Products

If a barcode is counted during physical inventory count but product information does not exist:

1. Mobile adds barcode to Unknown List.
2. Mobile may keep a local picture for reference, but does not send the picture to Qt.
3. Mobile still counts the quantity.
4. Mobile sends count result and barcode to Qt.
5. Qt asks user to complete product information.
6. User compares Qt barcode with mobile barcode and local picture.
7. After product information is completed, Qt can review the count.

Unknown products should not be applied to official stock until product info is completed.

---

## 23. Stock Adjustment

Stock adjustment is used to correct official stock quantity.

It can come from:

physical inventory count  
manual correction  
damage correction  
other

Adjustment fields:

transaction\_id  
barcode  
old\_quantity  
new\_quantity  
adjustment\_quantity  
reason  
user\_id  
user\_name\_snapshot  
transaction\_time  
created\_at  
note

Example:

System quantity: 100  
Real quantity: 97  
Adjustment quantity: -3  
Reason: physical\_inventory\_count  
User: staff001

---

## 24. Bluetooth Sync Rules

Bluetooth communication may fail.

Therefore, every sync must be safe and retryable.

---

## 24.1 Pending Sync Rule

If Bluetooth communication fails, the mobile app must save data as Pending Sync.

Pending Sync may include:

- unknown product list
- stock additions
- stock removals
- physical count sessions
- mobile product info suggestions

The mobile app must allow retry later.

---

## 24.2 ACK Rule

Qt must send an acknowledgement after successfully receiving and saving data.

The mobile app must not delete, reset, or mark data as synced until it receives ACK.

Example:

1. Mobile sends stock addition batch.
2. Qt receives and saves it.
3. Qt sends ACK.
4. Mobile marks batch as synced.
5. Mobile resets local pending quantity.

If ACK is not received:

- mobile keeps the batch as Pending Sync
- user can retry later

---

## 24.3 Duplicate-Send Prevention

Every sync batch must have a unique batch ID.

Example:

batch\_id: mobile-20260503-0001

Each item inside the batch should have a unique line ID.

Example:

batch\_id: mobile-20260503-0001

items:

- line\_id: 1  
barcode: 123456  
quantity: 10  
unit\_price: 5.00
- line\_id: 2  
barcode: 789000  
quantity: 3  
unit\_price: 8.00

If the same batch is sent twice, Qt must not apply it twice.

Qt should detect duplicate **batch\_id** values and ignore already-saved batches.

This prevents stock from being accidentally added or removed multiple times.

---

## 25. Sync Directions

| Data                              | Direction             | Main Owner                  |
|-----------------------------------|-----------------------|-----------------------------|
| User data                         | Qt → Mobile           | Qt                          |
| QR login token                    | Qt → Mobile / QR scan | Qt creates                  |
| Unknown List barcode and metadata | Mobile → Qt           | Mobile creates, Qt resolves |
| Unknown Product Picture           | Mobile only           | Mobile local reference only |
| Product Info                      | Qt → Mobile           | Qt                          |

|                                       |             |                     |
|---------------------------------------|-------------|---------------------|
| Mobile Product Info Suggestion        | Mobile → Qt | Qt approves         |
| Mobile Product Info Update Suggestion | Mobile → Qt | Qt approves         |
| Stock Addition                        | Mobile → Qt | Qt finalizes        |
| Stock Removal                         | Mobile → Qt | Qt finalizes        |
| Physical Count Session                | Mobile → Qt | Qt reviews          |
| Stock Adjustment                      | Qt only     | Qt finalizes        |
| Sync ACK                              | Qt → Mobile | Qt confirms receipt |

---

## 26. Validation Rules

The app should enforce:

- barcode must not be empty
- quantity must be greater than 0
- price must be greater than or equal to 0
- product name is required before official stock finalization
- product type is required before official stock finalization
- duplicate product records for the same barcode are not allowed
- stock quantity must not go below zero in MVP
- only active users can login
- only logged-in users can perform stock operations
- only admin users can access Admin Mode
- plain passwords must never be stored
- QR code must not contain password or personal sensitive data
- unknown product pictures must not be sent to Qt
- mobile-added or mobile-updated product information must be reviewed by Qt before becoming official

---

## 27. Mobile Local Database

The mobile app should store:

- local product information copy

local user login copy  
unknown product list  
unknown product pictures stored locally only  
pending stock additions  
pending stock removals  
pending physical count sessions  
temporary product info suggestions  
pending product info update suggestions  
sync batch history  
Bluetooth sync status  
current logged-in user session

The mobile database must work without internet and without Bluetooth.

---

## 28. PostgreSQL / Qt Database

The Qt app should store official data in PostgreSQL.

Recommended tables/concepts:

users  
user\_sync\_state  
user\_login\_tokens  
products  
unknown\_products  
product\_info\_suggestions  
stock\_quantities  
stock\_transactions  
physical\_count\_sessions  
physical\_count\_session\_items  
sync\_batches  
sync\_batch\_items

---

### 28.1 users

user\_id  
name  
password\_hash  
password\_salt  
birthday

mobile\_number  
role  
status  
created\_at  
updated\_at  
created\_by  
updated\_by  
last\_login\_at

Status values:

active  
inactive  
deleted

Recommended:

Use inactive instead of physical delete when user has history.

---

## 28.2 products

barcode  
product\_name  
country\_of\_manufacture  
product\_type  
default\_price  
details  
created\_at  
updated\_at  
created\_by  
updated\_by  
sync\_version

---

## 28.3 stock\_quantities

barcode  
current\_quantity  
updated\_at  
updated\_by

---

## 28.4 stock\_transactions



transaction\_id  
transaction\_type  
batch\_id  
barcode  
quantity  
unit\_price  
source\_location  
supplier\_name  
carried\_by  
reason  
destination\_type  
destination\_name  
destination\_contact  
destination\_note  
old\_quantity  
new\_quantity  
user\_id  
user\_name\_snapshot  
transaction\_time  
device\_id  
sync\_status  
created\_at

---

## 28.5 unknown\_products

unknown\_id  
barcode  
temporary\_product\_name  
temporary\_price  
pending\_quantity  
first\_seen\_at  
last\_seen\_at  
status  
created\_by\_user\_id  
resolved\_by\_user\_id  
resolved\_at  
sync\_status

Important:

Unknown product pictures are not stored in PostgreSQL.  
Unknown product pictures stay only in the mobile local database.

Status values:

unresolved  
resolved  
cancelled

---

## 28.6 physical\_count\_sessions

count\_session\_id  
started\_by\_user\_id  
started\_by\_user\_name\_snapshot  
started\_at  
ended\_at  
status  
device\_id  
sync\_batch\_id  
note

---

## 28.7 physical\_count\_session\_items

count\_session\_item\_id  
count\_session\_id  
barcode  
counted\_quantity  
system\_quantity\_at\_review  
difference\_quantity  
review\_status

---

## 28.8 sync\_batches

batch\_id  
batch\_type  
source\_device\_id  
created\_by\_user\_id  
created\_at  
received\_at  
ack\_sent\_at  
status

Batch types:

unknown\_list  
stock\_addition

stock\_removal  
physical\_count  
mobile\_product\_info\_suggestion  
mobile\_product\_info\_update\_suggestion  
user\_sync  
product\_info\_sync

Status values:

pending  
received  
processed  
acknowledged  
failed  
duplicate

---

## 28.9 user\_login\_tokens

For QR login:

token\_id  
user\_id  
token\_hash  
expires\_at  
created\_at  
used\_at  
status

Status values:

active  
used  
expired  
revoked

---

## 28.10 product\_info\_suggestions

This table stores product information added or updated from mobile before Qt approval.

suggestion\_id  
barcode  
suggestion\_type

suggested\_product\_name  
suggested\_country\_of\_manufacture  
suggested\_product\_type  
suggested\_default\_price  
suggested\_details  
source\_device\_id  
created\_by\_user\_id  
created\_by\_user\_name\_snapshot  
created\_at  
review\_status  
reviewed\_by\_user\_id  
reviewed\_at  
review\_note  
sync\_batch\_id

Suggestion types:

create  
update

Review status values:

pending\_review  
accepted  
edited\_and\_accepted  
rejected

---

## 29. UI Pages

### 29.1 Qt App Pages

The Qt app should include:

Login page  
Dashboard page  
Admin User Management page  
User Create/Edit page  
QR Code page  
Product Info page  
Product Info Suggestion Review page  
Unknown Product Review page  
Add Stock Confirmation page

Remove Stock Confirmation page  
Physical Inventory Count page  
Physical Count Review page  
Stock Quantity page  
Stock Transaction History page  
Sync Status page  
Bluetooth Connection page

---

## **29.2 Mobile App Pages**

The mobile app should include:

Login page  
QR Code Scan Login page  
Barcode Scan page  
Add Stock page  
Remove Stock page  
Physical Inventory Count page  
Unknown List page  
Product Info Entry page  
Pending Product Info Suggestions page  
Pending Sync page  
Current User page  
Bluetooth Connection page

---

## **30. Notifications**

### **30.1 Product Info Updated in Qt**

You updated some product information. Please sync it with the mobile app.

Button:

Send Product Info to Mobile

---

### **30.2 User Info Updated in Qt**

User information was updated. Please sync users with the mobile app.

Button:

Send User Data to Mobile

---

### **30.3 Mobile Pending Sync**

There is pending data waiting to sync with the Qt app.

---

### **30.4 Bluetooth Failure**

Bluetooth sync failed. The data has been saved as Pending Sync.

---

### **30.5 Successful Sync**

Sync completed successfully.

---

### **30.6 Unknown Product Before Stock Finalization**

This product is unknown. Please complete product information before finalizing stock.

---

### **30.7 Negative Stock Error**

Not enough stock available.

---

### **30.8 Mobile Product Info Suggestion Synced**

Product information suggestion was sent to the Qt app for review.

---

### **30.9 Qt Product Info Suggestion Received**

Mobile product information suggestions are waiting for review.

---

## **31. Example Workflows**

## 31.1 Admin Creates User

1. Admin logs in to Qt.
  2. Admin opens User Management page.
  3. Admin clicks Add User.
  4. Admin enters user\_id, name, password, birthday, mobile number, and role.
  5. Qt hashes the password.
  6. Qt saves the user to PostgreSQL.
  7. Qt activates Send User Data to Mobile.
  8. Admin syncs user data to mobile through Bluetooth.
  9. Mobile stores local login data.
- 

## 31.2 Mobile Login by Password

1. User opens mobile app.
  2. User enters user\_id and password.
  3. Mobile checks local synced user data.
  4. Mobile verifies password.
  5. If user is active, login succeeds.
  6. Mobile records current user session.
- 

## 31.3 Mobile Login by QR Code

1. User logs in successfully.
2. App generates QR code with user\_id and login token.
3. Mobile scans QR code.
4. Mobile verifies token using local synced token/user data.
5. If token is valid and user is active, login succeeds.

The QR code must not contain password, birthday, mobile number, or password hash.

---

## 31.4 Unknown Product Workflow

1. User logs in on mobile.
2. User scans barcode 888999.
3. Mobile cannot find product info.
4. Mobile adds barcode 888999 to Unknown List.
5. User may take product picture on mobile for local reference only.
6. Mobile does not send the picture to Qt.
7. Later, user sends Unknown List barcode and metadata to Qt.
8. Qt displays barcode as the main unknown item header.

9. User compares the barcode in Qt with the barcode and picture shown on mobile.
  10. Qt user enters product name, type, country, and default price.
  11. Qt saves official product info.
  12. Qt sends product info to mobile.
  13. Mobile updates local database.
  14. Mobile removes barcode 888999 from Unknown List.
  15. Mobile deletes local picture for barcode 888999.
- 

## **31.5 Add Known Product to Stock While Connected**

1. User logs in.
  2. Bluetooth is connected.
  3. User scans barcode 123456 on mobile.
  4. Mobile sends scan event to Qt.
  5. Qt increases temporary count.
  6. User enters quantity 20 or scans 20 times.
  7. User enters stock-entry price 5.00.
  8. User enters source information.
  9. User clicks “Add Products into Stock” in Qt.
  10. Qt saves stock addition transaction.
  11. Qt increases official stock quantity.
  12. Qt records user\_id and transaction time.
  13. Qt sends ACK.
  14. Temporary count is reset.
- 

## **31.6 Add Product to Stock While Disconnected**

1. User logs in on mobile.
  2. Bluetooth is not connected.
  3. User scans barcode 123456.
  4. Mobile finds product info locally.
  5. User enters quantity 20.
  6. User enters stock-entry price 5.00.
  7. Mobile saves this as Pending Sync.
  8. Later Bluetooth becomes available.
  9. Mobile sends pending stock addition to Qt.
  10. Qt saves transaction and increases stock quantity.
  11. Qt records user\_id and transaction time.
  12. Qt sends ACK.
  13. Mobile marks batch as synced.
  14. Mobile resets pending quantity.
-



## 31.7 Add Unknown Product to Stock

1. User scans barcode 777888.
  2. Mobile cannot find product info.
  3. Mobile adds product to Unknown List.
  4. Mobile may keep a local picture for reference only.
  5. Mobile does not send the picture to Qt.
  6. User enters quantity 10.
  7. User enters temporary stock-entry price 3.00.
  8. Mobile stores the count as pending.
  9. Mobile sends unknown barcode, metadata, and pending stock count to Qt.
  10. Qt asks user to complete product information.
  11. User compares Qt barcode with mobile barcode and local picture.
  12. User completes product information in Qt.
  13. Qt finalizes stock addition.
  14. Qt sends product info to mobile.
  15. Mobile removes item from Unknown List and deletes its local picture.
- 

## 31.8 Remove Product from Stock

1. User logs in.
  2. User scans barcode 123456.
  3. User enters quantity 5.
  4. User selects reason: sold.
  5. User enters outgoing destination, such as Customer / Shop Sale.
  6. User confirms removal.
  7. Qt checks available stock.
  8. Qt decreases official stock quantity by 5.
  9. Qt creates stock removal transaction.
  10. Qt records reason, destination, user\_id, and transaction time.
- 

## 31.9 Physical Inventory Count

1. User logs in.
2. User opens Physical Inventory Count page.
3. User scans existing products in the store.
4. App counts quantity for each barcode.
5. User submits the count session.
6. Qt compares counted quantity with official stock quantity.
7. Qt shows difference.
8. User confirms adjustment.
9. Qt creates stock adjustment transaction.

10. Qt updates official stock quantity.
11. Qt records user\_id and transaction time.

Example:

System quantity: 100  
Real counted quantity: 97  
Difference: -3

Adjustment:

transaction\_type = stock\_adjustment  
reason = physical\_inventory\_count  
old\_quantity = 100  
new\_quantity = 97  
adjustment\_quantity = -3

---

## 31.10 Mobile Adds or Updates Product Info

1. User logs in on mobile.
  2. User opens Product Info Entry page.
  3. User scans or selects a barcode.
  4. User enters or updates product information.
  5. Mobile saves it locally as a temporary product info suggestion.
  6. Mobile marks the suggestion as Pending Sync.
  7. When Bluetooth is available, mobile sends the suggestion to Qt.
  8. Qt shows the suggestion in Product Info Suggestion Review page.
  9. Qt user accepts, edits, or rejects the suggestion.
  10. If accepted, Qt saves official product information to PostgreSQL.
  11. Qt sends confirmed product info back to mobile.
  12. Mobile updates local product database.
  13. Mobile removes the pending suggestion.
- 

## 32. MVP Rules

For the first version, follow these rules:

Qt is the source of truth.  
Qt reads and writes PostgreSQL directly.  
Mobile does not connect directly to PostgreSQL.  
Mobile works offline with local copied data.  
Users are managed only in Qt Admin Mode.

Mobile login uses synced user data.  
QR code login must not expose password or sensitive personal data.  
Every stock operation records user\_id and timestamp.  
Unknown products can be counted but not finalized until product info is completed.  
Product pictures are stored only on mobile for unresolved unknown products.  
Product pictures are not sent to Qt.  
Qt Unknown List displays barcode as the main unknown item header.  
The user compares the Qt barcode with the barcode and local picture shown on mobile.  
Every stock addition, removal, and adjustment creates a transaction record.  
Stock removal should support outgoing destination information.  
Mobile-added or mobile-updated product information is synced to Qt as a suggestion when Bluetooth is available.  
Qt must review and approve mobile product information before it becomes official.  
Bluetooth sync must use Pending Sync, ACK, and batch\_id.  
Duplicate sync batches must not be applied twice.  
Physical Inventory Count does not directly change stock.  
Only confirmed Stock Adjustment changes official stock quantity.  
Stock quantity must not go below zero.  
Qt product information wins if there is a conflict.

---

## 33. Final Summary

The system works like this:

Qt app is the main offline management system.  
Qt connects directly to PostgreSQL.  
Mobile app is the offline scanning and counting tool.  
Bluetooth is used for sync between mobile and Qt.  
Users are created and managed only in Qt Admin Mode.  
Mobile receives synced user data for offline login.  
QR code login is allowed, but QR must not contain password or sensitive data.  
Product information is owned by Qt.  
Mobile can detect unknown products and send their barcodes to Qt.  
Unknown product pictures stay only on mobile and are used as local visual reference.  
Qt Unknown List displays barcode clearly so the user can compare it with the mobile picture and barcode.  
Mobile can add or update product information locally, but it must sync to Qt as a suggestion.  
Qt reviews and approves mobile product information before saving it as official product data.  
Stock additions and removals are confirmed by Qt.  
When products are removed from stock, the reason and outgoing destination can be recorded.  
Physical inventory count checks what currently exists in the store.  
Every stock change is saved as transaction history.

Every stock operation records user\_id and time.  
Nothing important is deleted or reset until Qt sends ACK.